

- Description

This module manages an elevator load monitoring system, ensuring that:

1. **Each person entering the elevator adds to the total weight.**
2. **If the total weight exceeds the elevator's capacity, an overload alarm is triggered.**
3. **If the weight is within limits, the elevator operates normally.**
4. **A log of all elevator trips, including weight and overload incidents, is maintained.**

This system ensures the safe operation of elevators by preventing overload conditions.

- Specification

module ElevatorLoadMonitoring

var maxCapacity, currentLoad: real

var passengerCount: integer

var tripLog: sequence of string

process enterElevator(passenger: string, weight: real)

functional scenario form:

 currentLoad ≤ maxCapacity && tripLog = tripLog ⊕ ("Overload Alert: " + passenger), Passenger entered, load updated.

 ||

 currentLoad > maxCapacity && tripLog = tripLog ⊕ ("Passenger Entered: " + passenger), Overload detected, alert triggered.

end process

process completeTrip()

functional scenario form:

 (passengerCount ≥ 0) && tripLog = tripLog ⊕ ("Trip Completed | Final Load: " + currentLoad), currentLoad = 0, passengerCount = 0, Elevator trip completed, load reset.

end process

end module

- Java Code

```
package code;
import java.util.*;
class Elevator {
    private double maxCapacity;
    private double currentLoad;
    private int passengerCount;
    private List<String> tripLog;
    public Elevator(double maxCapacity) {
        this.maxCapacity = maxCapacity;
    }
}
```

```

        this.currentLoad = 0.0;
        this.passengerCount = 0;
        this.tripLog = new ArrayList<>();
    }
    public void enterElevator(String passenger, double weight) {
        currentLoad += weight;
        passengerCount++;
        if (currentLoad > maxCapacity) {
            System.out.printf("ALERT! Overload detected! Passenger %s (%.1f kg) exceeded capacity!\n", passenger, weight);
            tripLog.add(String.format("Overload! Passenger: %s | Weight: %.1f kg | Total: %.1f kg (Exceeded!)", passenger, weight, currentLoad));
        } else {
            System.out.printf("Passenger %s (%.1f kg) entered. Current Load: %.1f kg\n", passenger, weight, currentLoad);
            tripLog.add(String.format("Passenger: %s | Weight: %.1f kg | Total: %.1f kg", passenger, weight, currentLoad));
        }
    }
    public void completeTrip() {
        System.out.printf("Elevator trip complete. Final Load: %.1f kg | Passengers: %d\n", currentLoad, passengerCount);
        tripLog.add(String.format("Trip Completed | Final Load: %.1f kg | Passengers: %d", currentLoad, passengerCount));
        currentLoad = 0.0;
        passengerCount = 0;
    }
    public void printTripLog() {
        System.out.println("\nElevator Trip Log:");
        for (String log : tripLog) {
            System.out.println(log);
        }
    }
}

public class c22_BuildingElevatorSystem {
    public static void main(String[] args) {
        Elevator elevator = new Elevator(600.0);
        // testcase-VT:
        //     elevator.enterElevator("Alice", 65.0);
        //     elevator.enterElevator("Bob", 80.0);
        //     elevator.enterElevator("Charlie", 75.0);
        //     elevator.enterElevator("David", 200.0);
        //     elevator.enterElevator("Eve", 220.0);
    }
}

```

```
//      elevator.completeTrip();

//      // testcase-FT:
//      elevator.enterElevator("??", -1.7976931348608188E308);
//      elevator.enterElevator("", 1.9490628022799996E289);
//      elevator.enterElevator("??mmm", -1.5439011628817536E308);
//      elevator.enterElevator("fhao0--", 0.0);
//      elevator.enterElevator("99AJFE", 3.800672464445999E290);
//      elevator.completeTrip();

//      // testcase-Z3:
//      elevator.enterElevator("Alice", 65);
//      elevator.enterElevator("Bob", 80);
//      elevator.enterElevator("Charlie", 75);
//      elevator.enterElevator("David", 200);
//      elevator.enterElevator("Eve", 220);
//      elevator.completeTrip();

//      // testcase-UVT:
//      elevator.enterElevator("Alice", 65.0);
//      elevator.enterElevator("Bob", 80.0);
//      elevator.enterElevator("Charlie", 75.0);
//      elevator.enterElevator("David", 200.0);
//      elevator.enterElevator("Eve", 220.0);
//      elevator.completeTrip();
//      elevator.printTripLog();
//  }
}
```

- Coverage report

1. Test case number: 5
2. V-Method testing branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c22_BuildingElevatorSystem.java	100.0 %	4	0	4
> c22_BuildingElevatorSystem		0	0	0
> Elevator	100.0 %	4	0	4

3. Vibration testing instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	213	16,525	16,738
▼ src	1.3 %	213	16,525	16,738
▼ code	1.3 %	213	16,525	16,738
▼ c22_BuildingElevatorSystem.java	98.6 %	213	3	216
> c22_BuildingElevatorSystem	90.9 %	30	3	33
> Elevator	100.0 %	183	0	183

4. Fuzz testing branch coverage: 75%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.4 %	3	839	842
▼ src	0.4 %	3	839	842
▼ code	0.4 %	3	839	842
▼ c22_BuildingElevatorSystem.java	75.0 %	3	1	4
> Elevator	75.0 %	3	1	4
> c22_BuildingElevatorSystem		0	0	0

5. Fuzz testing instruction coverage: 80.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.0 %	174	16,564	16,738
▼ src	1.0 %	174	16,564	16,738
▼ code	1.0 %	174	16,564	16,738
▼ c22_BuildingElevatorSystem.java	80.6 %	174	42	216
> Elevator	78.7 %	144	39	183
> c22_BuildingElevatorSystem	90.9 %	30	3	33

6. Z3 branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c22_BuildingElevatorSystem.java	100.0 %	4	0	4
> c22_BuildingElevatorSystem		0	0	0
> Elevator	100.0 %	4	0	4

7. Z3 instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	213	16,525	16,738
▼ src	1.3 %	213	16,525	16,738
▼ code	1.3 %	213	16,525	16,738
▼ c22_BuildingElevatorSystem.java	98.6 %	213	3	216
> c22_BuildingElevatorSystem	90.9 %	30	3	33
> Elevator	100.0 %	183	0	183

8. Unimproved V-method branch coverage: 100%

Element	Coverage	Covered Branches	Missed Branches	Total Branches
▼ VibrationTestingTool	0.5 %	4	838	842
▼ src	0.5 %	4	838	842
▼ code	0.5 %	4	838	842
▼ c22_BuildingElevatorSystem.java	100.0 %	4	0	4
> c22_BuildingElevatorSystem		0	0	0
> Elevator	100.0 %	4	0	4

9. Unimproved V-method instruction coverage: 98.6%

Element	Coverage	Covered Instructions	Missed Instructions	Total Instructions
▼ VibrationTestingTool	1.3 %	213	16,525	16,738
▼ src	1.3 %	213	16,525	16,738
▼ code	1.3 %	213	16,525	16,738
▼ c22_BuildingElevatorSystem.java	98.6 %	213	3	216
> c22_BuildingElevatorSystem	90.9 %	30	3	33
> Elevator	100.0 %	183	0	183